

System Architecture

Detailed architecture document for the Medical FAQ project.

1. Overview

The system is split into five layers: users, frontend, API, services, and data. Every layer is independently deployable through docker-compose. The whole stack is designed to run on a single 4 vCPU 8 GB VM so a small clinic or a university lab can host it without cloud spend.

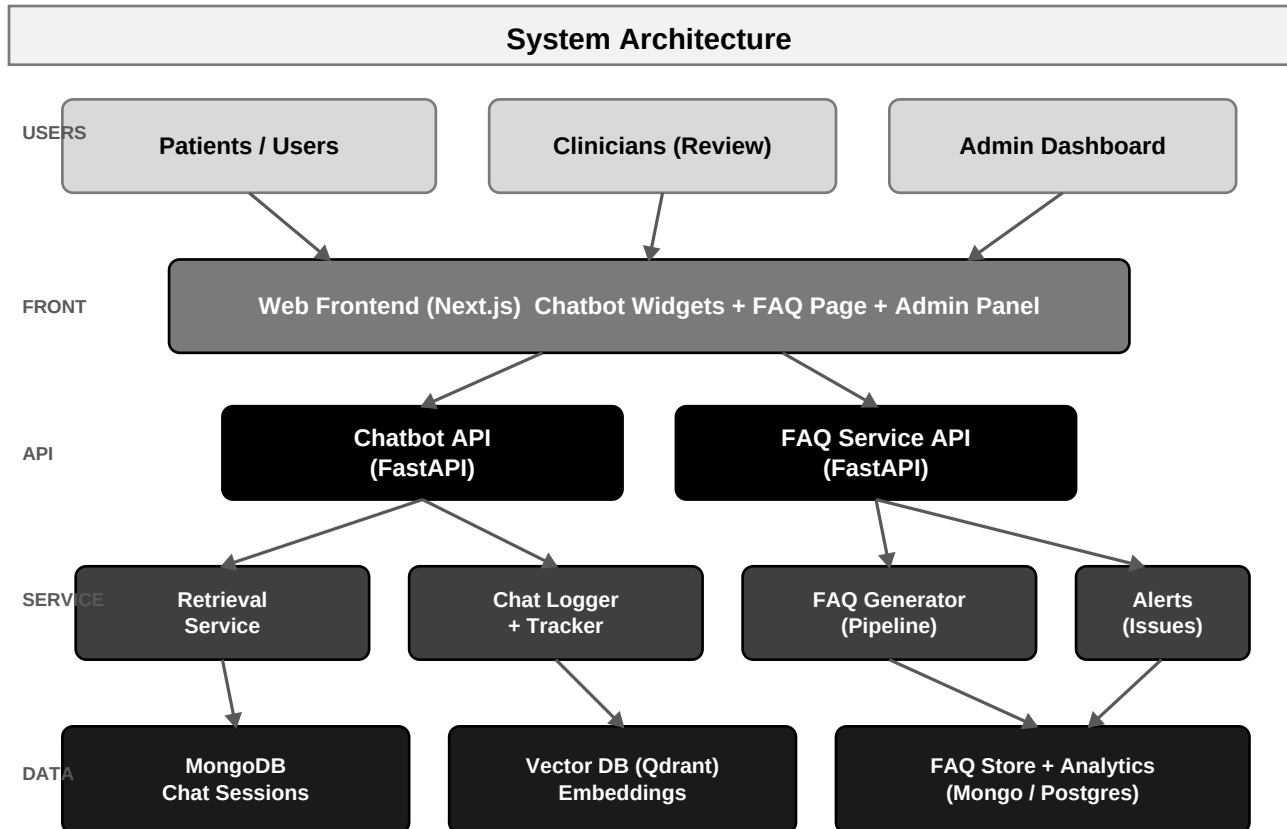


Fig 1. Five-layer system view.

2. Frontend Layer

Next.js 15 (React 19) handles all browser-facing pages. Routes are file-based under `src/app/`. Three pages are public (landing, chat, FAQ) and one is admin-only. Components are kept small and single-purpose so each member can iterate without stepping on others' work.

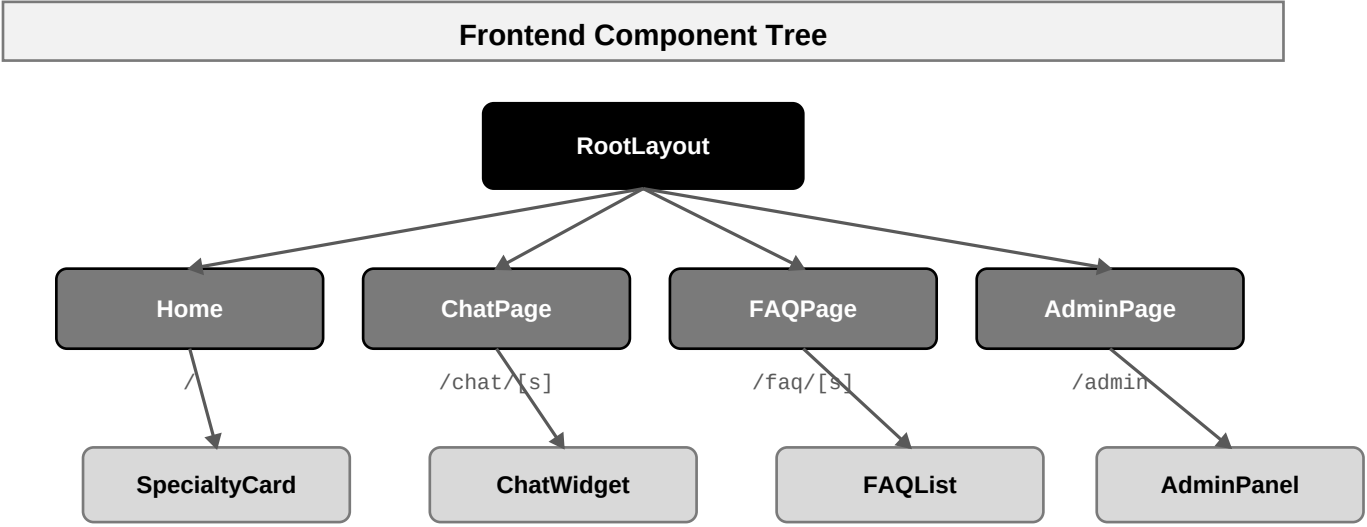


Fig 2. Frontend component tree (routes connect to components).

Path	Component	Responsibility
<code>/</code>	Home	Lists the three specialties as cards.
<code>/chat/[s]</code>	ChatPage + ChatWidget	Streams chat turns to the API, displays answers.
<code>/faq/[s]</code>	FAQPage + FAQList	Lists approved auto-built FAQs for the specialty.
<code>/admin</code>	AdminPage	Approve / edit / reject candidates, view trending issues.

3. Backend Layer

FastAPI exposes three routers: /chat, /faq, and /admin. Routers are thin and delegate to a service layer (retrieval, PHI scrub, chat logger). The service layer is the only place that talks to MongoDB or Qdrant, so swapping a store later stays local.

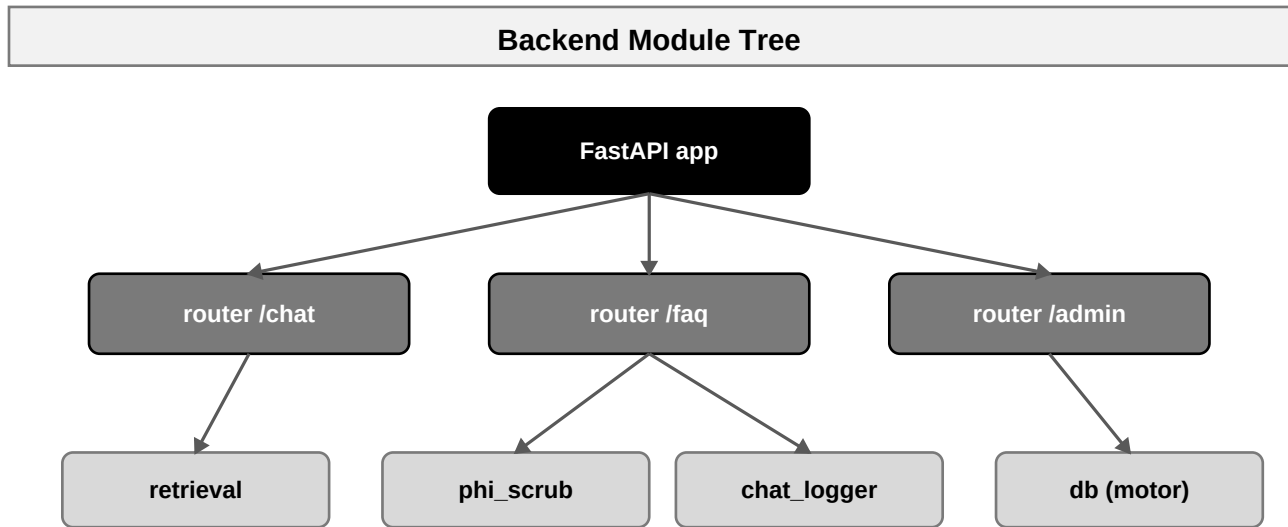


Fig 3. Backend module tree (FastAPI -> routers -> services).

Endpoint	Method	Purpose
/health	GET	Liveness probe used by docker / NGINX.
/chat	POST	Take a user message, log it, return retrieval answer.
/faq/{specialty}	GET	List approved FAQs for the specialty.
/admin/candidates/{specialty}	GET	List candidate FAQs awaiting clinician review.
/admin/approve/{faq_id}	POST	Mark a candidate as approved.
/admin/edit/{faq_id}	POST	Edit a candidate's question or answer.
/admin/reject/{faq_id}	POST	Mark a candidate as rejected.

4. Sequence Flows

Two key flows: a user chat request, and a scheduled pipeline run. Diagrams below trace each call across components.

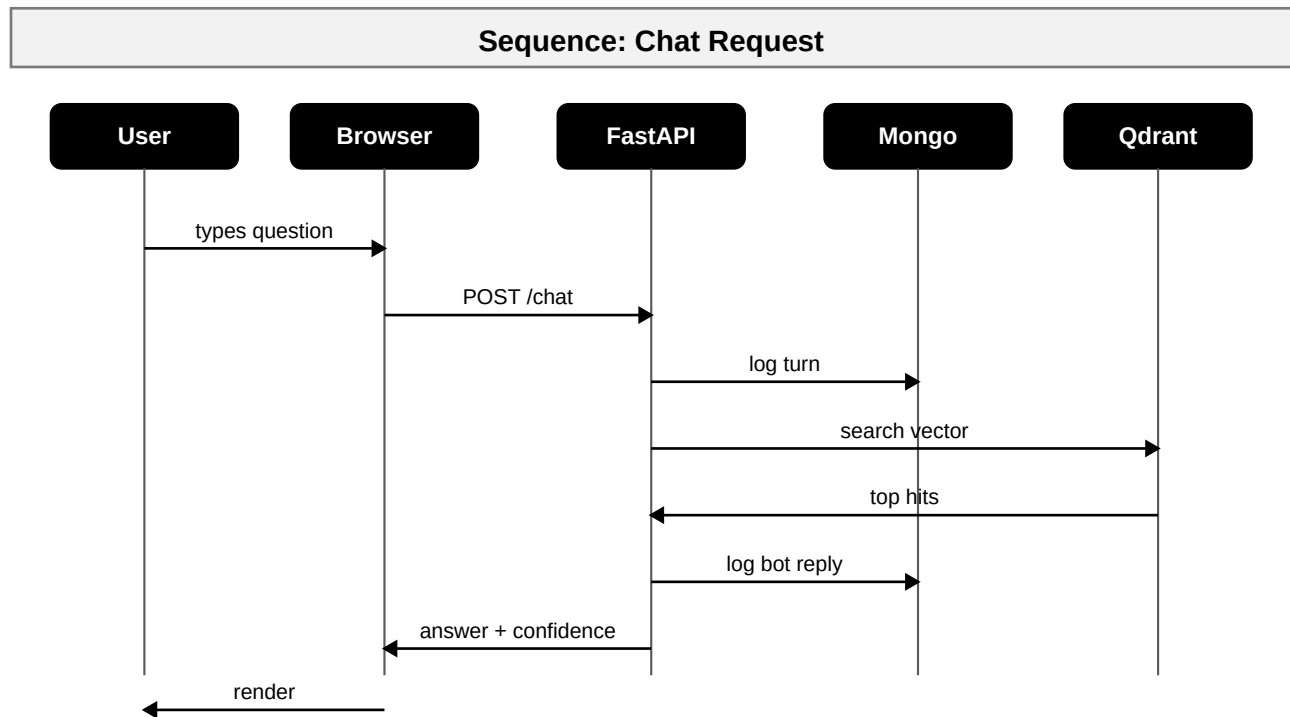


Fig 4. Chat request flow: user types question to bot reply.

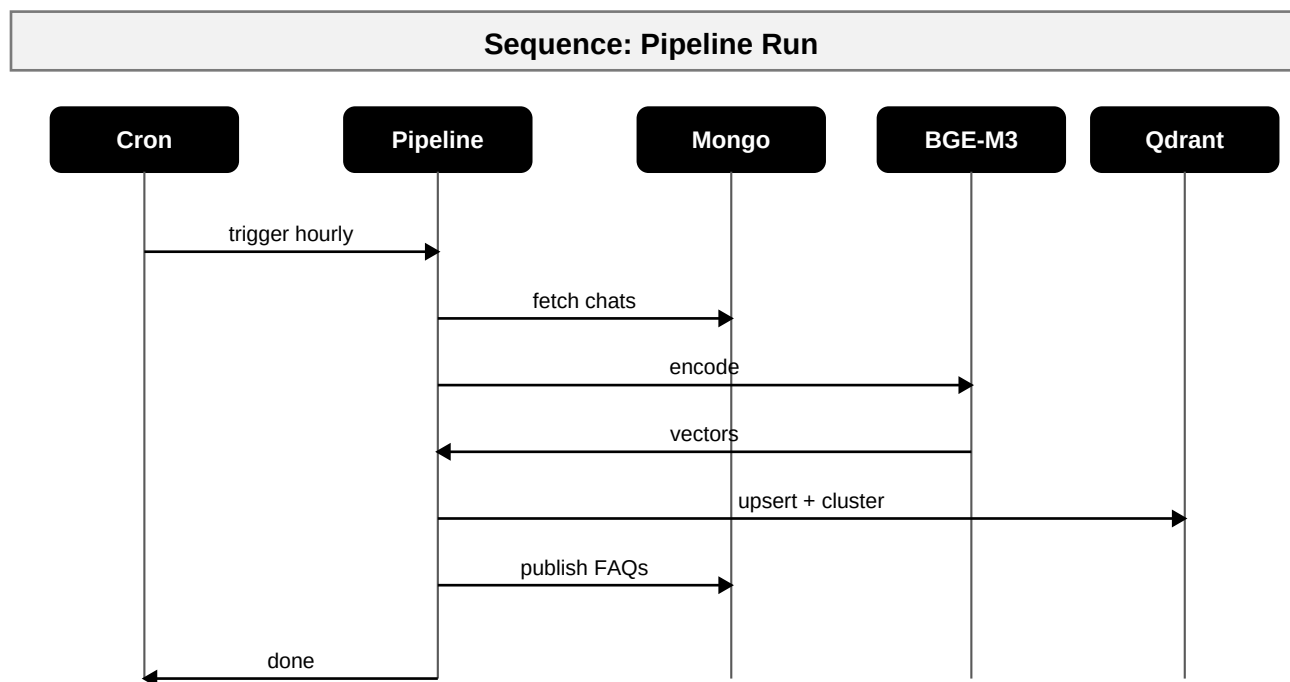


Fig 5. Pipeline flow: cron triggers fetch, encode, cluster, publish.

5. Deployment

Single VM. One docker-compose file brings up Mongo, Qdrant, and Redis. The Next.js and FastAPI processes run via systemd or under Docker, depending on the operator's preference. NGINX terminates TLS and routes `/api/*` to FastAPI and the rest to Next.js. Celery worker plus Celery beat handle the scheduled pipeline runs.

Deployment Topology (single VM)

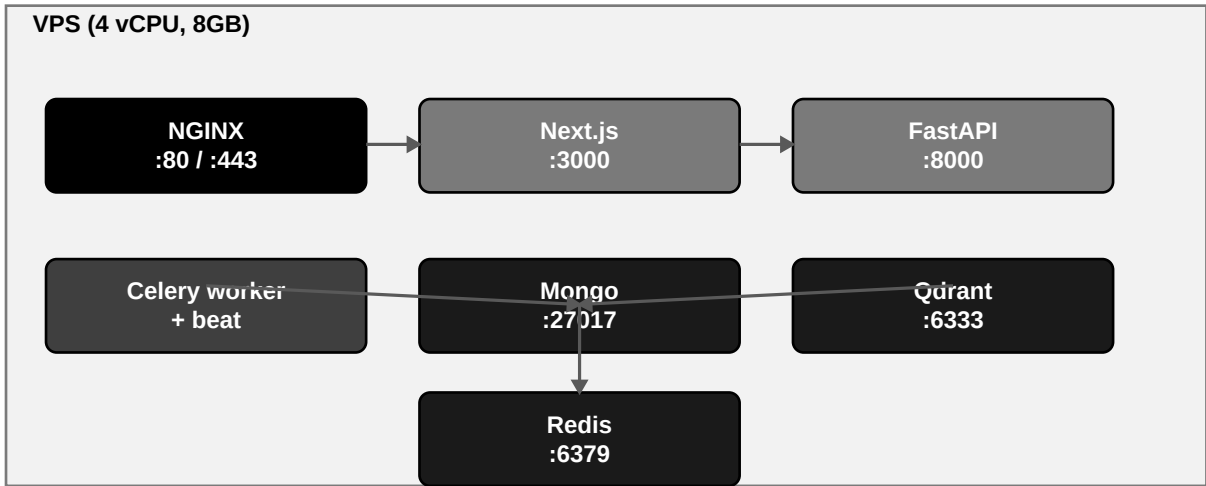


Fig 6. Single-VM deployment topology with ports labelled.

Component	Port	Restart Policy
NGINX	80, 443	always (systemd)
Next.js	3000	always (systemd or docker)
FastAPI (uvicorn)	8000	always (systemd or docker)
Celery worker	(internal)	always (docker)
MongoDB	27017	unless-stopped (docker)
Qdrant	6333, 6334	unless-stopped (docker)
Redis	6379	unless-stopped (docker)

6. Trade-offs and Decisions

- No LLM. Trade: simpler, traceable answers and zero hallucination risk. Cost: chatbot cannot synthesise novel answers; it can only return what is already in the index.
- Single VM. Trade: easy to operate by a 5-person student team. Cost: vertical scaling only.
- Mongo + Qdrant split. Trade: write-heavy chat logs go to Mongo, latency-sensitive search goes to Qdrant. Cost: two stores to operate.
- Celery scheduling instead of an event stream. Trade: simpler to reason about for a 16-week project. Cost: not real-time.
- Approval workflow before publishing. Trade: clinician safety. Cost: slower to surface FAQs publicly.